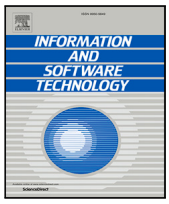
Information and Software Technology 194 (2026) 108077



Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof





Test automation with selenium: A survey

Boni García [a,*], Filippo Ricca [b], Maurizio Leotta [b], Mario Munoz-Organero [a]

[a] Universidad Carlos III de Madrid, Madrid, Spain
[b] Dipartimento di Informatica, Bioingegneria, Robotica e Ingegneria dei Sistemi (DIBRIS), Università di Genova, Genova, Italy

ARTICLE INFO

Keywords:
Browser automation
Test automation
E2E testing
Selenium
AI-assisted tools

ABSTRACT

Context: Selenium is a widely used tool for end-to-end (E2E) web testing. However, it is often criticized for brittleness, slowness, and flakiness. In parallel, newer frameworks and Artificial Intelligence (AI) are reshaping the test automation landscape.
Objectives: This study aims to investigate current practices, challenges, and emerging trends in Selenium-based test automation.
Methods: We designed and executed a large-scale online survey targeting software professionals who use Selenium. The questionnaire covered technical practices, tooling, AI usage, perceived challenges, and competing tools.
Results: A total of 88 complete responses were analyzed using descriptive statistics and thematic coding. The results show that Selenium remains the dominant tool for regression and functional testing, primarily using the Page Object Model (POM) pattern. The most reported challenges are related to assertability, asynchrony, and brittleness. AI tools like ChatGPT are gaining traction for test generation. Playwright is the most prominent alternative.
Conclusion: While Selenium is recognized as a cornerstone in many automation workflows, its limited native test-specific features present a significant drawback. The findings indicate an increasing demand for testing-focused improvements within the Selenium ecosystem, as well as for enhanced integration with AI-driven development tools.

1. Introduction

Selenium[1] is an umbrella open-source project that provides browser-automation capabilities. The core component of this project is Selenium WebDriver (typically referred to simply as Selenium), which is a browser automation library that allows practitioners to simulate user interactions with real browsers (e.g., Chrome, Firefox, Edge, etc.) programmatically through a binding language (such as Java, JavaScript, Python, Ruby, or C#) [1].

End-to-end (E2E) testing is a type of black-box testing that validates whether an application behaves as expected from the user's perspective across the entire system, including the frontend, backend, and any external services. Automated E2E tests are crucial in ensuring the quality and reliability of modern web applications. Hence, Selenium has been widely adopted [2] for implementing automated tests for web applications and, more generally, automation workflows. However, contrary to popular belief, Selenium is not a testing framework itself but a general-purpose browser automation library, as it lacks built-in testing features such as assertions, test runners, and reporting tools. Furthermore, problems with Selenium E2E testing have been reported, including brittleness and flakiness [3]. Finally, the test automation landscape has undergone considerable evolution in recent years. New tools, such as Cypress or Playwright, have entered the market, offering built-in testing features and an improved developer experience [4].

This paper presents the results of an online survey of professionals to gain a deeper understanding of how the community uses Selenium for E2E testing. The goal of this study is to provide up-to-date knowledge of Selenium's role in web testing, usage patterns, strengths and shortcomings, and its position relative to emerging tools. Moreover, this survey aims to analyze how Selenium users use Artificial Intelligence (AI) tools (such as AI agents and chatbots) to facilitate the development of Selenium-based tests [5].

This paper contributes to the understanding of Selenium-based end-to-end (E2E) testing practices in the following ways:

- It provides a detailed investigation into how practitioners leverage Selenium and its ecosystem to implement automated E2E

* Corresponding author.
E-mail address: boni.garcia@uc3m.es (B. García).
[1] https://www.selenium.dev/.

https://doi.org/10.1016/j.infsof.2026.108077
Received 22 July 2025; Received in revised form 10 February 2026; Accepted 10 February 2026
Available online 12 February 2026
0950-5849/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

tests. This includes insights into usage scenarios (e.g., functional and regression testing), technical implementation strategies (e.g., locator patterns, synchronization techniques), supporting infrastructure (e.g., browsers and drivers), and auxiliary tools and frameworks.
- It identifies and thoroughly analyzes the most significant challenges encountered by practitioners using Selenium, encompassing technical hurdles, feature gaps, and critical pain points that could lead to adopting alternative tools.
- It explores the application of AI-powered tools, such as generative AI (GenAI) and other intelligent assistants, to support and enhance the Selenium-based testing process, detailing their perceived benefits and limitations.
- It examines alternative E2E testing tools that practitioners consider as supplements or replacements for Selenium, analyzing their respective advantages, drawbacks, and the specific contexts in which they are favored.

Collectively, the study's findings offer a comprehensive perspective on the current landscape of Selenium-based testing. They illuminate real-world practices, pervasive challenges, and emerging trends (notably the increasing influence of AI), thereby furnishing valuable, actionable insights for researchers, practitioners, and tool developers alike.

This study builds on several years of research by our team. It complements our previous work analyzing Selenium and its ecosystem [6], as well as the challenges of end-to-end (E2E) testing with Selenium [3]. Specifically:

- In 2022, we surveyed 78 highly skilled industry professionals [3] to investigate the most significant challenges in E2E testing, focusing on Selenium. That study provided key insights into: (a) the primary difficulties developers face daily with Selenium, and (b) practical strategies practitioners use to mitigate issues like test flakiness and fragility.
- An earlier survey [6], conducted in 2019 with 72 participants, mapped the community's use of Selenium and its ecosystem. This study was organized into seven categories: Selenium foundations, test development, system under test, test infrastructure, other frameworks, community, and personal experience.

The current survey builds on and updates our previous research, while also exploring new dimensions, including AI's role in enhancing E2E testing practices with Selenium.

The remainder of this paper is structured as follows. Section 2 outlines the relevant aspects of the survey conducted, such as goals, research questions, sample identification, questionnaire design, survey preparation and execution, and analysis methodology. Section 3 presents the results and findings. Section 4 discusses the results obtained. Section 5 examines the inevitable threats to validity. Section 6 discusses related work, and finally Section 7 concludes the paper, presenting ideas for future work.

2. Study definition

We use a survey as an instrument to take a snapshot of the current state of practice regarding test automation with Selenium. To design this survey, we drew inspiration from previous surveys [3,6] and followed the guidelines provided by Kitchenham and Pfleeger [1].

The survey was conducted through the following six steps: (1) select objectives (or goals), (2) transform goals into research questions, (3) questionnaire design, (4) sample and evaluate the questionnaire using pilot executions, (5) execute survey, and (6) results analysis and packaging. First, we designed this survey to understand the following goals:

G1 The use of Selenium and its ecosystem to implement automated E2E tests for Web applications.

G2 The challenges of E2E testing with Selenium.

G3 The AI-assisted tools used to facilitate the process of test automation with Selenium.

G4 The use of alternative tools (not belonging to the Selenium ecosystem) for E2E test automation.

2.1. Research questions

Given the above goal, the survey aims to address the following research questions:

RQ1 *How do practitioners use Selenium and its ecosystem to implement automated E2E tests?* To answer this question, we will analyze the use cases fulfilled by practitioners using Selenium (such as functional testing, regression testing, etc.), the technical aspects of the Selenium-based development (such as programming languages, waiting and locator strategies, and design patterns), the requirement infrastructure (browsers and drivers), and the helper tools (like unit or E2E testing frameworks).

RQ2 *What are the most relevant challenges as perceived by practitioners in Selenium-based E2E testing?* We will analyze the common difficulties that Selenium users face, together with the possible solutions to these problems. Also, we will try to identify the missing features that Selenium users demand and the critical issues that could lead to abandoning Selenium in favor of another tool.

RQ3 *What do practitioners use the most relevant AI-powered tools to ease the process of E2E testing with Selenium?* We will analyze generative AI (GenAI) tools and other AI-powered testing tools that can be used to facilitate the use of Selenium. In addition, we examine the perceived benefits and potential drawbacks of using such tools.

RQ4 *What do practitioners use the most relevant alternative tools for E2E testing?* We will analyze the landscape of potential tools competing in the browser and test automation arena, along with the perceived benefits and possible drawbacks of using such tools.

2.2. Population and sampling strategy

We began by identifying the target population for our study, specifically focusing on software engineers involved in E2E testing with Selenium. This group typically includes developers, testers, and Quality Assurance (QA) professionals. We then established a framing population composed of these practitioners, who were invited to respond to our questions regarding the target items. To select participants, we used nonprobabilistic sampling techniques, including convenience and snowball sampling [1].

Specifically, the sampling was performed in four different ways: (i) asking for participation during the international conference SeleniumConf and AppiumConf 2025,[2] held in Valencia, Spain, from 26 to March 28 2025; (ii) asking for participation during the 18th IEEE International Conference on Software Testing, Verification and Validation (ICST) 2025,[3] held in Naples, Italy, from March 31 to April 4 2025; (iii) sending invitations to the authors' social networks, including X and LinkedIn; and (iv) selecting contacts from the industrial networks of the two research units involved (Madrid and Genova). Our approach included elements of snowball sampling, as participants

[2] https://seleniumconf.com/.
[3] https://conf.researchr.org/home/icst-2025.

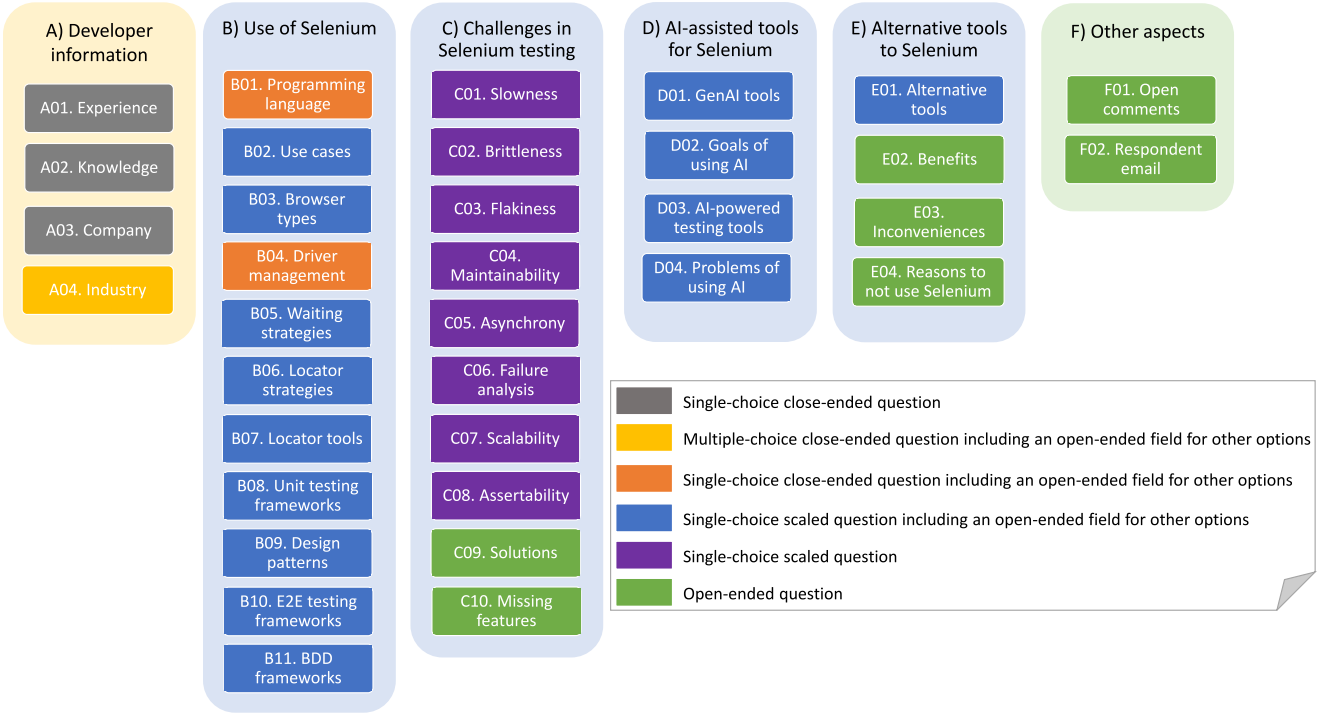


Fig. 1. Survey items per categories and question types.

were encouraged to share the survey with peers in their professional networks, enabling broader reach within the community.

We decided to collect data through an online questionnaire created using Google Forms.[4] Compared to paper- or email-based questionnaires, web-based questionnaires provide respondents with easier data entry and offer researchers a more streamlined, less error-prone data collection process [7]. Moreover, studies have shown that web-based questionnaires generally achieve higher response rates [8].

2.3. Survey preparation and execution

The process of preparing, administering, and collecting questionnaire data comprises four main steps.

Preparation and design of the questionnaire. We started by drafting a preliminary version of the questionnaire. To identify potential issues, we conducted three pilot studies with software professionals [1]. Based on the feedback received, we refined the questionnaire to improve clarity and ensure the instrument's validity for online deployment.

Online deployment. After finalizing the content, the questionnaire was implemented using Google Forms. This platform facilitated automated data collection and participant invitations.

Invitation to participate. The organizations were sampled as described in Section 2.2. Participation was requested at selected conferences (SeleniumConf/AppiumConf 2025 and ICST 2025) and through broadcast messages on the authors' social networks (X and LinkedIn), which included a link to the questionnaire and a brief cover letter. The letter explained the objectives of the study, its relevance to the participants, and the importance of individual contributions [1]. For directly selected contacts, invitations were sent via email after identifying the appropriate contact persons. No material incentives were offered; however, to encourage participation, we promised to share a report summarizing the analyses and results with all respondents.

Data analysis and packaging. Once data collection concluded, we carried out the analyses described in Section 2.5. The instruments, collected data, and results were then compiled into a replication package.

[4] https://forms.gle/WWt3KX8z3JGG1NQf6.

2.4. Questionnaire design

The questionnaire was specifically designed to align with the study objectives. Fig. 1 summarizes the final instrument comprising a total of 35 items and organized into six sections.

(A) Developer information. This section aims to characterize respondents' development experiences. In particular, it collects years of experience, rate of knowledge of Selenium, company size, and organization type (industry, academia, or other).
(B) Use of Selenium. This section collects information on how Selenium and its ecosystem are utilized for performing E2E automated testing, including programming languages, use cases, infrastructure, coding strategies, helper tools, and design patterns [6].
(C) Challenges in Selenium testing. This section compiles information on the most significant challenges of using Selenium in automated testing, as reported in [3]: slowness, brittleness, flakiness, maintainability, asynchrony, failure analysis, scalability, and assertability.
(D) AI-assisted tools for test automation with Selenium. This section compiles information on the AI tools respondents use to support Selenium script development.
(E) Alternative tools for browser and test automation. This section compiles information about potential alternative tools, such as Cypress, Puppeteer, or Playwright [4], along with their benefits and drawbacks compared to Selenium.
(F) Open comments. This section closes the questionnaire by collecting two optional items: open comments and the email address of the respondent who wants to receive a copy of the findings.

Table 1 summarizes all these questions as they were presented to the respondents. All questions in our survey were mandatory. For identification purposes, each section is labeled with an uppercase letter, and each item is tagged using the corresponding section code (A to F) followed by a two-digit sequential number (e.g., B04 refers to the fourth item in the second section). As illustrated in Fig. 1, there are different types of questions, namely:

- Single-choice closed-ended questions. The respondents were presented with a list of mutually exclusive options and instructed to select the one that best reflected their opinion or situation. This format was used to capture categorical data on respondents' experience, knowledge, and company.
- Multiple-choice closed-ended questions, including an open-ended field for other options. These items allowed respondents to select multiple options from a predefined list. This format was particularly suitable for capturing multifaceted experiences related to the respondent's work (industry, academia, or other).
- Single-choice closed-ended question including an open-ended field for other options. These items present a list of mutually exclusive options, including an "Other" field for an alternative answer not included in the predefined list. This type of question is used for the programming languages (Java, JavaScript, C#, Ruby, Python) and the driver management tools (third-party, built-in, not required, unknown) [9,10].
- Single-choice scaled questions with an open-ended field for other options. In this format, respondents were presented with a matrix in which each row represented a distinct item, and a consistent set of columns allowed for the selection of a single response per row. Respondents indicated the importance of the different rows using a 5-point Likert-type scale (ranging from "Never" to "Always"). This type of question (marked in blue in Fig. 1) is used for multiple items, such as Selenium use cases, browser types, or all the questions related to tools (testing frameworks, BDD, GenAI, etc.). Table 2 presents the content of each row in all the questions using this format. To accommodate responses not covered by the predefined options, the final row included an "Other (please specify)" field, enabling participants to input unlisted alternatives.
- Single-choice scaled question. Respondents indicated their level of agreement or perception on a 5-point Likert-type scale (from "Fully disagree" to "Fully agree"). These elements facilitated the quantification of subjective evaluations regarding the challenges faced in Selenium development.
- Open-ended questions. To allow for the expression of unstructured qualitative insights, several open-ended questions were included. These items provided respondents the opportunity to elaborate on their experiences or clarify responses in their own words.

2.5. Analysis methodology

The nature of this survey is descriptive, as it aims to represent certain conditions or factors within a population in terms of their frequency and impact [1]. Therefore, we applied descriptive statistics and presented our findings through charts. Additionally, we employed thematic coding to analyze the open-ended responses [11]. This qualitative method involves reading through each response, identifying meaningful segments, and assigning concise labels ("codes") that reflect key ideas. Similar codes were then grouped into higher-level themes. This enabled us to systematically extract patterns and insights from unstructured text while preserving the respondent's intent. Thematic coding was used to analyze challenges (e.g., solutions in C09) and missing features (C10), as well as the perceived benefits and drawbacks of alternative tools (E02–E04). For replication, the anonymized raw data and the replication package are available online.[5]

3. Results

This section first presents the characteristics of the population sample. Then, it presents the results and findings related to the four research questions.

[5] https://github.com/bonigarcia/test-automation-selenium-survey.

Table 1
Questions included in the survey.

Id	Question
A01	How many years have you been working in the field of test automation?
A02	How would you rate your knowledge of Selenium?
A03	What is the size of your company?
A04	Where do you work? (industry, academia, other)
B01	Which primary programming language do you use with Selenium?
B02	For which use cases do you typically employ Selenium?
B03	What type of browsers do you use for your Selenium scripts?
B04	How do you manage the browser drivers (e.g. chromedriver, geckodriver, etc.)?
B05	What waiting strategy do you implement in your Selenium scripts?
B06	What locator strategy do you implement in your Selenium scripts?
B07	Which tool do you use to create locators?
B08	Which unit testing framework do you use in conjunction with Selenium?
B09	Which design patterns do you use for developing Selenium scripts?
B10	Which End-to-End (E2E) testing frameworks of the Selenium ecosystem do you use?
B11	Which Behavior-Driven Development (BDD) framework do you use with Selenium?
C01	Are the following problems (related to Selenium) relevant, in your opinion? Slowness - Selenium scripts are slow to run, i.e., take a long time to complete
C02	Brittleness - Selenium scripts are brittle, i.e., fragile when web app evolution occurs
C03	Flakiness - Selenium scripts are flaky, i.e., they can break randomly without a clear reason
C04	Maintainability - The activities required to keep Selenium scripts operative after deployment are troublesome
C05	Asynchrony - Selenium scripts must manage asynchronous interactions, which are problematic to handle
C06	Failure analysis - Troubleshooting Selenium scripts is difficult since the failure causes might be unclear
C07	Scalability - Managing a large Selenium suite is complex (e.g., when using parallel execution)
C08	Assertability - Selenium script assertions are challenging to create
C09	How do you try to solve the previous problems?
C10	What are the features that you consider missing in the current Selenium API?
D01	Which generative AI tools do you use to support the development of Selenium scripts?
D02	For which aspects of software testing and test automation do you use some AI-powered tool?
D03	Which AI-assisted tools have you used for test automation?
D04	What potential problems do you see when using AI-assisted tools for test automation?
E01	What alternative tools to Selenium and its ecosystem do you use for test automation?
E02	What are the benefits of these alternative tools compared to Selenium and its ecosystem?
E03	What are the inconveniences of these alternative tools compared to Selenium and its ecosystem?
E04	What could be the reason for stopping the use of Selenium and its ecosystem, and using some alternative tools?
F01	Optionally, provide any further comments on Selenium or any other topic you want to mention.
F02	Provide your email address (if you want to receive a copy of our research findings)

3.1. The sample

The survey was available online from March 26 to May 30, 2025. We collected 88 complete responses to our survey. Due to the sampling methods used, it is not possible to estimate how many people our invitation messages reached. Consequently, it is not possible to compute

Table 2
Content of the scaled questions that included an open field for other options.

Id	Row content
B02	Functional testing, Regression testing, Cross-browser testing, Performance testing, Security testing, UI/UX testing, Accessibility testing, Simulated mobile testing, Web scraping, Other
B03	Local browsers, Remote browser (e.g., Selenium Grid), Cloud browsers (e.g., Sauce Labs, LambaTest, etc.), Dockerized browsers, Other
B05	None, Implicit wait, Explicit wait, Fluent wait, Manual wait, Other
B06	By id, By name, By XPath, By CSS selector, By class name, By link text, By partial link text, By tag name, Using relative locators, Other
B07	Browser Developer Tools, SelectorsHub, ChroPath, Selenium IDE, Other
B08	JUnit, TestNG, Spock, JSUnit, PyTest, Unittest, PyUnit, Mocha, Jasmine, Gauge, Jest, NUnit, VSTest, RSpec, Test::Unit, Other
B09	Page Object Model (POM), Screenplay, Factory, Singleton, Facade, Other
B10	CodeceptJS, FluentSelenium, FluentLenium, Healenium, Helium, Robot Framework, Selenide, SeleniumBase, Watir, Nightwatch.js, WebDriverIO, Other
B11	Cucumber, FitNesse, JBehave, Jasmine, Capybara, Serenity BDD, Other
D01	ChatGPT, GitHub Copilot, Google Gemini, DeepSeek, Amazon CodeWhisperer, Other
D02	Test cases generation, Test cases prioritization, Test cases maintenance, Exploratory testing, Test oracle generation, Predictive analysis, Visual testing, Performance testing, Self-healing tests, Other
D03	Testim, Applitools, Functionize, Mabl, ReTest, Testers.ai, Katalon Studio, Other
D04	Accuracy (e.g., false positives or false negatives), Privacy concerns (e.g., sharing sensitive or proprietary information), Security vulnerabilities (e.g., expose secrets), Explainability (e.g., hard-to-understand AI-generated code), Over-reliance on AI (e.g., reduce critical thinking and problem-solving skills), Other
E01	Cypress, Puppeteer, Playwright, TestCafe, Other

the response rate. This limitation is a common problem for large-scale online surveys [12].

To provide an overview of the respondents' backgrounds, we visualized responses to the first section of the questionnaire (i.e., questions A01–A04) using bar charts. Fig. 2 illustrates the years of experience of participants in test automation. In particular, 31% of respondents have extensive experience of more than 10 years (red). The largest group, with responses of 33%, has between 5 and 10 years of experience (orange). Furthermore, 28% of participants reported 1–4 years of experience (blue), while a small number reported less than 1 year (green). These results indicate that we successfully reached a significant number of highly experienced professionals.

The bar chart shown in Fig. 3 reinforces that we successfully invited participants who consider their Selenium knowledge substantial. Specifically, 48% of respondents rated their experience as "Advanced" (blue), while 39% identified as "Intermediate" (green). Finally, the smaller portion, 14%, rated their knowledge as "basic" (green).

Fig. 4 presents an overview of the companies in which the participants work. Most of the responses (over 60%, blue) came from practitioners working in large companies (with 250 or more employees). The second-largest group consists of those employed in medium-sized companies (50-249 employees), accounting for 21% of responses (orange). A smaller proportion of the participants reported working in small companies (10–49 employees) and micro companies (1–9 employees), representing 10% (orange) and 9% (green) of the total, respectively. Overall, these results indicate that the survey primarily captured perspectives from practitioners working in medium-to-large organizations,

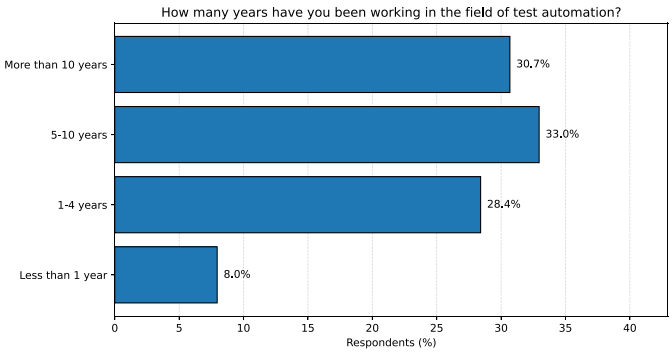


Fig. 2. Developer Information Results: Years of experience (A01).

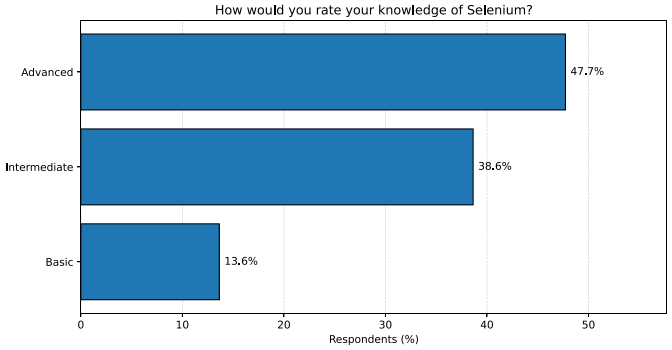


Fig. 3. Developer Information Results: Selenium knowledge (A02).

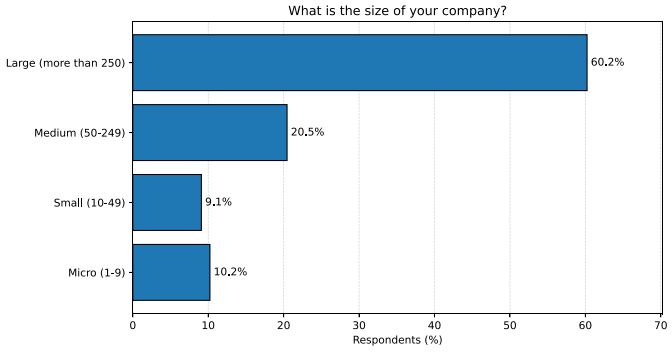


Fig. 4. Developer Information Results: Company size (A03).

where structured and large-scale test automation practices are more likely to be established.

Finally, Fig. 5 presents the distribution of respondents' workplaces. The graph indicates that the vast majority of respondents, more than 77% (blue), are employed in industry. Smaller proportions are found in academia (12%; green) and in other sectors (10%; orange), accounting for 22% of the total. Some respondents used the "Other" field to specify particular industry sectors such as consulting, e-commerce, or entertainment. Others used this field to describe roles that do not fit neatly into traditional industry or academic categories, including open-source contributions and positions in public institutions. These results suggest that this study's findings predominantly reflect industrial practice while incorporating viewpoints from academia and other professional contexts.

3.2. RQ1. Selenium and its ecosystem

A key element in analyzing how practitioners use Selenium and its ecosystem for test automation is understanding the main use cases of

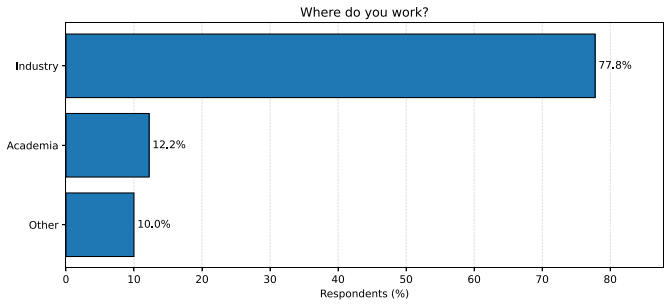


Fig. 5. Developer Information Results: Workplace (A04).

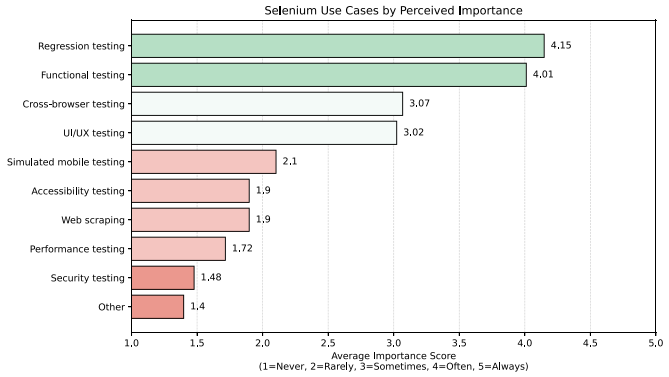


Fig. 6. Use of Selenium Results: Use cases (B02).

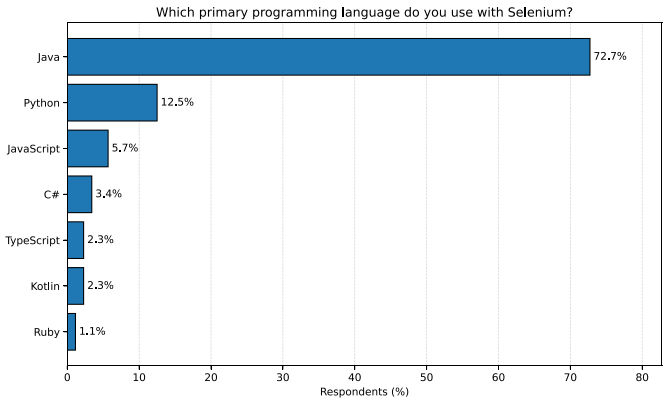


Fig. 7. Use of Selenium Results: Programming language (B01).

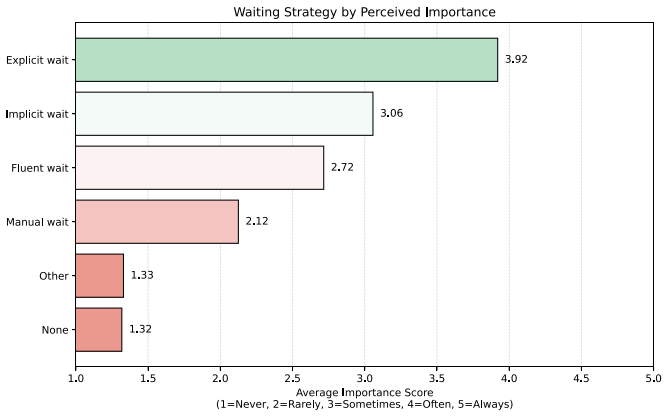


Fig. 8. Use of Selenium Results: Waiting strategies (B05).

Selenium. This information is gathered in item B02 of the survey, and its results are provided in Fig. 6. This bar chart illustrates the perceived importance of different Selenium use cases, as measured by the average usage scores of survey respondents. Responses were weighted on a scale with higher scores indicating greater importance ("Never" = 1, "Rarely" = 2, "Sometimes" = 3, "Often" = 4, and "Always" = 5). This way, the averages reflect the frequency with which Selenium is employed for each task, ranging from least (1) to most critical (5). The data shows that Regression Testing (4.15) and Functional Testing (4.01) are the most highly perceived use cases, indicating they are mostly "Often" to "Always" important in Selenium workflows. Cross-browser testing (3.07) and UI/UX testing (3.02) fall in the middle, suggesting they are "Sometimes" important. Lower-scored use cases, such as Simulated mobile testing (2.1), Accessibility testing (1.9), Web scraping (1.9), Performance testing (1.72), Security testing (1.48), and Other (1.4), are perceived as "Rarely" to "Sometimes" necessary. Although the "Other" category is the least relevant according to the data, this open-ended field revealed interesting secondary use cases, including automating daily or repetitive tasks in a web browser, keeping websites alive, and monitoring and observability.

The second group of elements related to Selenium use concerns the technical aspects of Selenium-based development. In this group, we find items B01 (programming language), B05 (waiting strategy), B06 (locator strategy), and B09 (design patterns). Regarding B01, as shown in Fig. 7, Java is the dominant language among the respondents, with an adoption rate of 73%, likely due to its long-standing compatibility with Selenium and strong unit testing frameworks (e.g., TestNG, JUnit) [13]. Python reflects its popularity and growing adoption in test automation [14], ranking second in this survey (13%). JavaScript is another popular language in test automation, although it ranks third with a score of 6% in this survey. Finally, C#, TypeScript, Kotlin, and Ruby are the least common languages for the respondents to this survey.

Another essential aspect of Selenium script development is the waiting strategy, which synchronizes script execution with dynamic web content to ensure elements are ready before interaction. The management of waiting strategies has been reported in the literature as a crucial mechanism for avoiding test failures and flakiness [15,16]. Hence, item B05 gathers the preferred waiting strategies in this survey. This way, Fig. 8 shows that "Explicit wait" is the most valued strategy, with a score of 3.92, indicating that it is used frequently and considered very important. "Implicit wait" (3.06) and "Fluent wait" (2.72) are moderately important. Surprisingly, even though manual waiting is a well-known anti-pattern [13], some respondents still use it (2.12). "None" (1.32) is the least important, suggesting most respondents find some waiting strategy essential. Regarding the open-ended field "Other" (1.33), several participants reported creating custom waits by combining different strategies.

Similarly, choosing the right locator strategies (i.e., methods for identifying and interacting with web elements on a page) is essential for creating reliable and maintainable Selenium scripts [17]. Thus, Fig. 9 shows the results concerning locator strategies (item B06). In light of the results, the highest-rated strategy is by id (3.86), likely due to its reliability and performance, which is almost equally significant as XPath (3.81), which offers flexibility for complex DOM structures [18]. Then, we find some moderately used strategies by CSS selector (3.26), by name (2.86), and by class name (2.64). The remaining strategy (relative locators, link text, tag name, and partial link text) is identified as less important. Finally, regarding the "Other" open-ended field, although it is rarely used, it provides examples of less-standard strategies, such as accessibility locators based on WAI-ARIA[6] and CSS locators built based on custom data (`data-t` HTML attribute). This confirms

[6] https://www.w3.org/WAI/standards-guidelines/aria/.

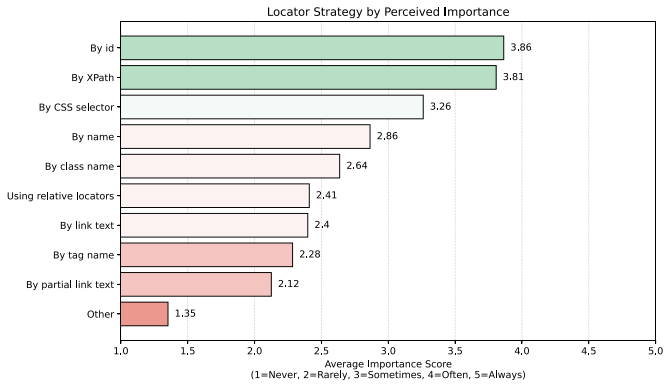


Fig. 9. Use of Selenium Results: Locator strategies (B06).

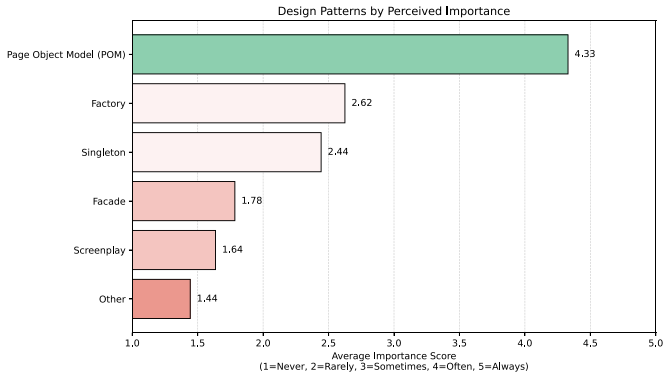


Fig. 10. Use of Selenium Results: Design patterns (B09).

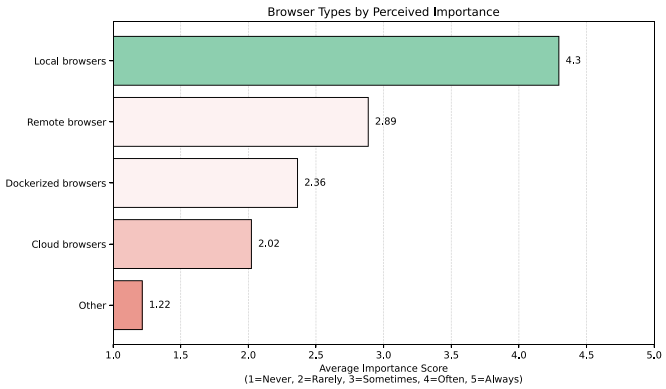


Fig. 11. Use of Selenium Results: Browser types (B03).

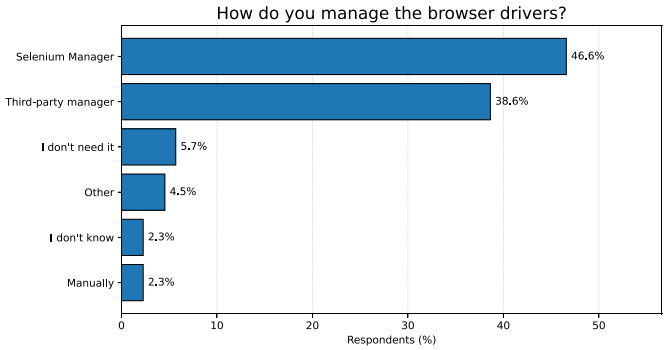


Fig. 12. Use of Selenium Results: Driver management (B04).

that practitioners favor locator strategies that balance robustness and flexibility, prioritizing stability over expressiveness.

The last technical aspect analyzed in this survey is design patterns, i.e., the best-practice coding strategies that help practitioners create more maintainable, scalable, and robust Selenium scripts [19]. In this regard, and as illustrated in Fig. 10, the Page Object Model (POM) is the preferred design pattern in test automation [20], with an average importance score of 4.33, indicating that it is frequently used. All other patterns trail significantly, with Factory (2.62) and Singleton (2.44) receiving moderate attention. Patterns like Facade (1.78) and Screenplay (1.64) are perceived as less critical or less commonly used. Finally, in the "Other" field (1.44), we find many other design patterns, including Strategy, Command, Builder, Decorator, Observer, and Dynamic Proxy. These results reinforce the dominance of the POM as the de facto standard for structuring Selenium-based test code.

The following group of elements related to this RQ concerns the infrastructure required by the Selenium scripts: browsers (item B03) and drivers (item B04). Regarding browsers, Fig. 11 shows a strong preference for local browsers, which received the highest average importance score of 4.3, suggesting they are the primary choice for test execution. Remote browsers (e.g., Selenium Grid) follow at 2.89, reflecting moderate use for distributed testing. Dockerized browsers (2.36) and cloud browsers (2.02), such as the browser-as-a-service provided by companies like Sauce Labs,[7] BrowserStack,[8] or LambdaTest,[9] are less commonly used, likely due to their cost. "Other" browser types scored the lowest at 1.22, with Kubernetes and headless browsers specified in this open-ended field. These results indicate a clear preference for simpler, locally executed testing infrastructures over more complex or costly alternatives. Then, Fig. 12 reveals that nearly half of the respondents (47%) use Selenium Manager, making it the most common method for handling browser drivers (i.e., chromedriver for Chrome, geckodriver for Firefox, etc.). Selenium Manager is a built-in tool included in each Selenium release starting from 2022, which automates driver management [21]. Third-party managers (such as WebDriverManager[10] for Java or webdriver-manager[11] for Python) are also popular, with 39% of users utilizing them. Only a small fraction of respondents manage drivers manually (2%), report that they don't need it (6%), or don't know how it is managed (2%). An additional 5% do not use any other methods. This distribution highlights a strong preference for automated or built-in solutions over manual driver management.

The final group of items regarding the use of Selenium concerns tooling. In this group, we find items B07 (locator tools), B08 (unit testing frameworks), B10 (E2E testing frameworks), and B11 (BDD frameworks). Fig. 13 illustrates the results regarding locator tools, revealing a dominant preference for Browser Developer Tools, with an average importance score of 3.97. This result indicates that built-in development tools in web browsers (e.g., DevTools[12] in Chrome or Firefox Developer Tools[13] in Firefox) are almost universally used for creating web element locators in Selenium development. In contrast, specialized tools like SelectorsHub[14] (1.58), Selenium IDE[15] (1.51),

[7] https://saucelabs.com/.
[8] https://www.browserstack.com/.
[9] https://www.lambdatest.com/.
[10] https://bonigarcia.dev/webdrivermanager/.
[11] https://pypi.org/project/webdriver-manager/.
[12] https://developer.chrome.com/docs/devtools.
[13] https://firefox-source-docs.mozilla.org/devtools-user/.
[14] https://selectorshub.com/.
[15] https://www.selenium.dev/selenium-ide/.

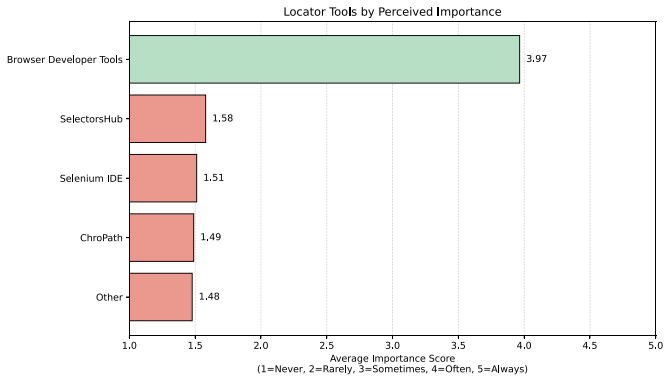


Fig. 13. Use of Selenium Results: Locator tools (B07).

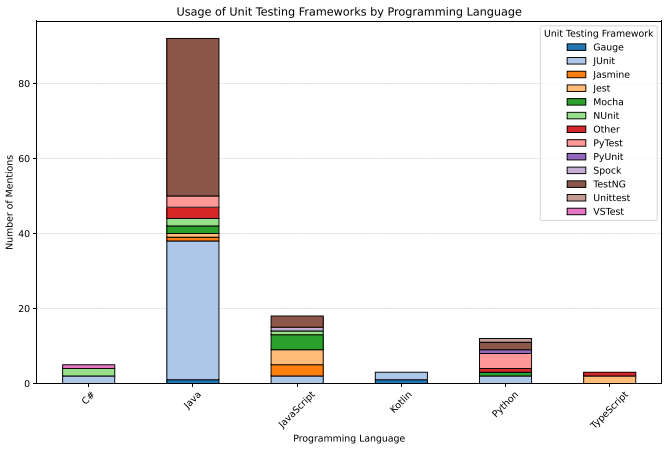


Fig. 14. Use of Selenium Results: Unit testing frameworks (B08).

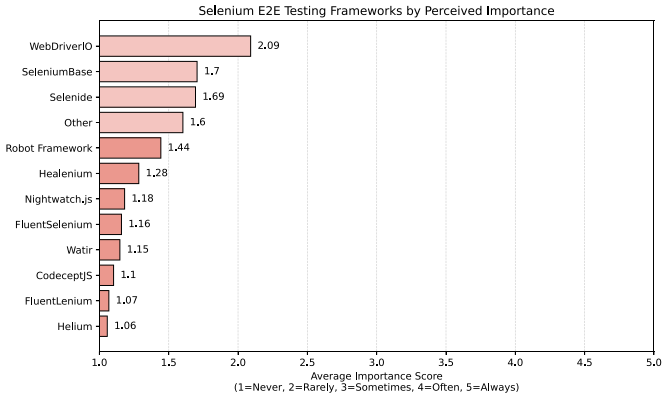


Fig. 15. Use of Selenium Results: E2E testing frameworks (B10).

or ChroPath[16] (1.49) received low scores, suggesting they are rarely used or valued. The "Other" field also gets a low score (1.48), but interestingly, this open-ended field revealed different ways to create locators, including browser extensions like CSS Selector Helper,[17] Ranorex Selocity,[18] xPath Finder,[19] and XPath Tester.[20] Finally, a couple of respondents claimed to use a custom tool for creating locators.

Selenium is a general-purpose browser automation library; therefore, it is necessary to use a unit testing framework alongside Selenium to create comprehensive E2E tests [13]. Therefore, item B08 analyses the unit testing framework used by respondents. Since unit testing frameworks are linked to a programming language, Fig. 14 illustrates these results using a stack bar chart. This way, each stack displays the unit testing framework with the relevant importance (i.e., those selected as "Always" and "Often" by respondents). Regarding Java, JUnit, and TestNG are the most prominent. JavaScript shows diverse uses across Jest, Mocha, and Jasmine. Python primarily shows PyTest and Unittest. NUnit and VSTest are used in C#. Finally, Kotlin and TypeScript have minimal representation, mostly limited to JUnit (for Kotlin) and Jest or PyTest (for TypeScript). Regarding the "Other" field, it is worth noting that some respondents reported using a custom unit-testing framework. In contrast, others used high-level testing frameworks covered in different parts of this study (such as Cucumber, Robot Framework, and WebdriverIO).

While Selenium itself is the core library for browser automation, its ecosystem has grown to include additional frameworks and tools to improve test development [6]. Hence, item B10 explores how commonly respondents use E2E testing frameworks on top of Selenium WebDriver. Fig. 15 indicates that no single framework significantly dominates, with WebDriverIO[21] receiving the highest average importance score of 2.09, suggesting it is used occasionally but not widely. SeleniumBase[22] (1.7) and Selenide[23] (1.69) follow closely, while all other frameworks (including Robot Framework,[24] Healenium,[25] and Nightwatch.js[26]) score below 1.5, indicating limited adoption. The low scores across the board suggest that no Selenium-based frameworks have achieved a broad consensus or widespread usage within the community. Regarding the "Other" options, it is notable that the prevalence of custom or internal Selenium-based frameworks is high.

Finally, item B11 examines the BDD frameworks. BDD is a collaborative software development approach that utilizes natural language to describe test scenarios, typically employing a Gherkin syntax (e.g., Given-When-Then) [22]. These scenarios are then linked to automated test code, often using Selenium for browser interaction, helping ensure that development aligns closely with business requirements and user expectations [6]. Fig. 16 shows that Cucumber[27] stands out as the only BDD tool with notable importance in this study, receiving an average score of 3.02, indicating it is sometimes used too often. All other tools, including Serenity BDD[28] (1.36), JBehave[29] (1.23), and FitNesse[30] (1.18), score below 2, indicating rare or minimal use. These results suggest that Cucumber is the clear leader and de facto standard, while most other tools see limited traction. Regarding "Other" alternatives, some respondents highlighted SpecFlow,[31] Reqnroll,[32] Behave,[33] and other frameworks using BDD style (concretely Robot Framework, Gauge, and Mocha).

16 https://addons.mozilla.org/firefox/addon/chropath-for-firefox/.
17 https://chromewebstore.google.com/detail/css-selector-helper/gddgceinofapfodcekopkjjelkbjodin.
18 https://chromewebstore.google.com/detail/ranorex-selocity/ocgghcnnjekfpbmafindjmijdpopafoe.
19 https://chromewebstore.google.com/detail/xpath-finder/ihnknokegkbpmofmafnkoadfjkhlogph.
20 https://chromewebstore.google.com/detail/xpath-tester/cneomjecgakdfoeehmmmoiklncdiodmh.
21 https://webdriver.io/.
22 https://seleniumbase.io/.
23 https://selenide.org/.
24 https://robotframework.org/.
25 https://healenium.io/.
26 https://nightwatchjs.org/.
27 https://cucumber.io/.
28 https://serenity-bdd.github.io/.
29 https://jbehave.org/.
30 https://fitnesse.org/.
31 https://github.com/browserstack/specflow-browserstack.
32 https://reqnroll.net/.
33 https://github.com/behave/behave.

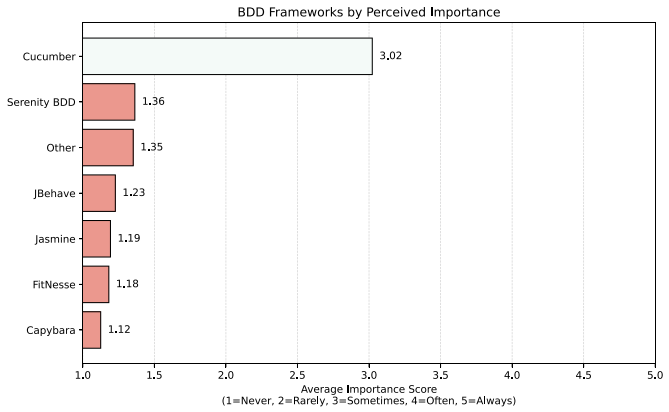


Fig. 16. Use of Selenium Results: BDD frameworks (B11).

RQ1 Summary: The primary use cases of Selenium are regression and functional testing, mainly for driving local browsers and automatically managed drivers. Regarding coding, the preferred strategy for waiting is explicit, and locators are primarily created using IDs and XPath, with the browser developer tools as a helper. The preferred programming language is Java, typically used with TestNG or JUnit for unit testing. Cucumber is the preferred BDD framework. Selenium-based E2E frameworks have not achieved widespread usage within the community. The most popular design pattern is POM.

3.3. RQ2. Selenium challenges

To investigate the key challenges in developing Selenium-based E2E tests, we drew upon the empirically validated framework presented in [3], which systematically identified and addressed the major pain points in Selenium E2E testing. This study highlighted the following primary challenges:

- **Asynchrony** refers to the challenge of handling dynamic web elements or events that occur at unpredictable times, making it difficult to synchronize test execution with the application state.
- **Brittleness** refers to the tendency of tests to fail unpredictably due to minor changes in the application (e.g., UI structure, timing, or environment) rather than actual defects, making tests unreliable.
- **Flakiness** refers to tests that produce inconsistent results (passing and failing under the same conditions) due to non-deterministic factors like race conditions, environmental instability (e.g., network latency), or poorly synchronized tests.
- **Assertability** refers to the challenge of reliably verifying expected outcomes, including non-DOM-based assertion types (e.g., visual properties checking).
- **Scalability** refers to the challenge of efficiently maintaining and executing tests as the codebase, test suite, or system under test complexity grows.
- **Slowness** refers to the challenge of excessively long test execution times, impeding fast feedback loops in CI/CD pipelines and reducing testing efficiency.
- **Failure analysis** (also called troubleshooting) refers to the challenge of efficiently diagnosing the root cause of test failures, which can stem from application defects, test script issues (e.g., flaky assertions), environmental factors (e.g., network latency), or synchronization problems (e.g., race conditions).
- **Maintainability** refers to the challenge of keeping test scripts adaptable, readable, and easy to update as the application evolves without requiring excessive effort or refactoring.

Table 3
Average and median ratings of selenium challenges, from 1 (Fully disagree) to 5 (Fully agree).

Issue	Average	Median
Assertability	3.43	3
Asynchrony	3.24	3
Brittleness	3.15	3
Failure analysis	3	3
Flakiness	2.93	3
Maintainability	2.9	3
Scalability	2.83	3
Slowness	2.66	3

Table 3 summarizes the results of the perceived importance of each challenge in our survey (items C01 to C08). In light of these results, the most strongly agreed-upon challenges, based on average ratings, are Assertability (3.43), Asynchrony (3.24), and Brittleness (3.15), indicating common frustrations with test validation, timing issues, and test stability. Other challenges, such as Flakiness (2.93), Maintainability (2.9), Scalability (2.83), and Slowness (2.66), are perceived as less severe but still relevant. The median scores remain consistent across all challenges at 3, suggesting general agreement on their moderate impact.

Comparing the global results presented in Table 3, there is a difference of almost a point between the highest-rated (Assertability) and the lowest-rated challenge (Slowness). This gap is not substantial, but it is significant given a five-point scale. Therefore, to obtain a more accurate description of the results, Fig. 17 provides a deeper look into the perceptions of the users of the top-4 Selenium challenges (Assertability, Asynchrony, Brittleness, and Failure Analysis). Looking at this chart, it is important to note that the largest group in Assertability and Asynchrony, with around 31% of responses, is “Not sure”. This result indicates some uncertainty about these challenges. For example, Assertability might not be a relevant issue for those users who rely entirely on classical DOM-based assertions. Additionally, Asynchrony may not be a challenge for users who do not test web applications with asynchronous or dynamic content. Regarding Assertability, the largest group (31%) selected “Not sure”, followed by “Agree” (28%), indicating mild concern but some uncertainty. However, this challenge has a stronger overall agreement, with over 45% of responses leaning toward “Agree” or “Fully agree”, reflecting widespread awareness of synchronization issues. Then, Brittleness stands out, with the highest percentage (33%) agreeing, which reinforces it as a widely acknowledged problem in test stability. Finally, the responses regarding Failure Analysis are more evenly distributed, with “Disagree”, “Not sure”, and “Agree” each accounting for approximately 25%–30%, indicating a mixed sentiment and less consensus.

Fig. 18 further explores the remaining challenges. Regarding Flakiness, most respondents lean toward “Agree” (27%) and “Not sure” (25%), indicating moderate concern about test instability. Then, a large portion (28%) disagrees that Maintainability is a significant issue, although around 27% agree, indicating a split opinion. This fact can be attributed to the specific development practices (e.g., best practices and design patterns) employed by the respondents. Interestingly, Scalability ranks the highest agreement (34%), reflecting strong user consensus that scaling Selenium tests is challenging. Finally, opinions about Slowness are varied, with approximately 28% disagreeing and a similar portion unsure; fewer users rated it as a top concern (“Fully agree”, 5%). This fact can be attributed to the specific web applications being tested or the development strategies employed by respondents.

The next question related to this RQ concerns the solutions respondents attempt to use to address these challenges. The results of this open-ended question (item C09) are summarized in Table 4. As shown in this table, some solutions address different challenges. For example, using an explicit waiting strategy is a potential solution to

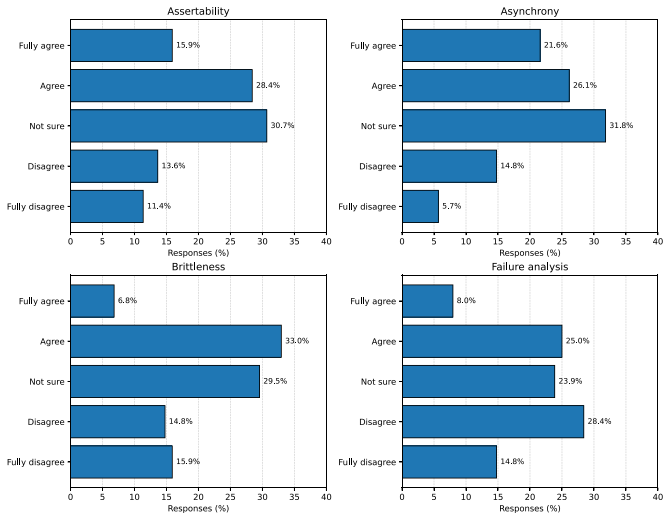


Fig. 17. Details of the answers distribution for the challenges ranked in the top four positions.

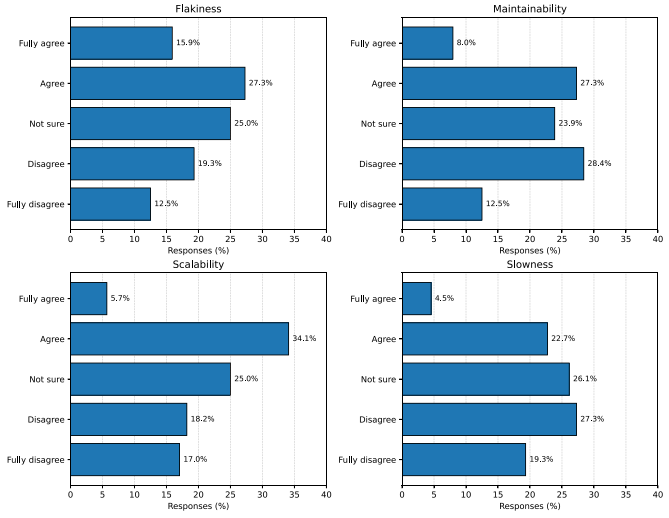


Fig. 18. Details of the answers distribution for the challenges ranked from position 5 to position 8.

the challenges of Asynchrony, Flakiness, and Slowness. Then, a robust and stable locator strategy has been highlighted to address problems related to Brittleness, Maintainability, and Scalability. In particular, the following locator strategies were highlighted by respondents:

- Unique IDs, when possible.
- Resilient XPath generated by specific tools.
- Custom HTML attributes, such as `data-testid`, specifically for testing purposes.
- CSS locators using value-oriented strategies like `contains()` and partial matches.

Then, the POM design pattern was selected to address Maintainability and Scalability issues, as it separates page structure from test logic, making updates easier and tests more reusable. Finally, the use of specific tools was proposed to ease Failure Analysis, Flakiness, or Maintainability.

Finally, item C10 investigates the features that respondents consider missing in the Selenium API. Through thematic coding, we classify the answers to this open-ended question into three groups:

Table 4
Common solutions for well-known Selenium challenges.

Challenge	Solution
Assertability	- Integrate computer vision models or testing tools (e.g., Applitools, Sikuli, Argos) to enhance visual and layout testing capabilities.
Asynchrony	- Use an explicit waiting strategy that provides granular control over delays. - Use state machines to manage test flow by defining clear states (e.g., "Login", "Home") and transitions based on conditions (such as element visibility) to ensure each step only runs when the previous one is completed. - Follow good practices when using promises in JavaScript, such as ensuring each promise is fully resolved before proceeding to the next step and handling potential errors within the same promise using the `.catch()` method.
Brittleness	- Use of a robust and stable locator strategy. - Collaborate closely with development and product teams to understand the app behavior better, plan for changes early, and write more stable tests.
Failure analysis	- Use proper logging in the system under test. - Use Docker to create an isolated test environment. - Analyze the exception stack traces to discover the cause of the failure. - Manually inspect the browser during test execution. - Screencasting (i.e., recording a video of the browser activity). - Screenshot comparison. - Use a custom tool for reporting, such as ReportPortal[a].
Flakiness	- Use an explicit waiting strategy that provides granular control over delays. - Run several times failing tests to detect if a test is flaky. - Use specific tools, such as Healenium for auto-healing locators (i.e., automatically fix broken locators at runtime).
Maintainability	- Use of a robust and stable locator strategy. - Use the POM design pattern (for reusability). - Split complex tests into smaller pieces. - Utilize refactoring features in IDEs (e.g., IntelliJ IDEA) to expedite maintenance operations.
Scalability	- Reduce the number of parallel test executions on the same machine. - Running headless browsers in parallel. - Use of a robust and stable locator strategy. - Use the POM design pattern (for reusability).
Slowness	- Use an explicit waiting strategy that provides granular control over delays. - Avoid manual waits (e.g., thread sleep). - Avoid implicit waiting as much as possible, since delays apply globally. - Disable non-essential content (e.g., animations) in test environments to ensure optimal performance. - Check specific UI elements (e.g., loading indicators) to assess the page state.

[a] https://reportportal.io/.

- Missing features in the Selenium API. In this group, we find automatic/smart waiting, multimedia and visual assertions (image, video, sound), integrated reporting, accessibility testing, native mobile support, Oauth2 authentication, mutation testing features, native parallelization, observability features, retries mechanism for flaky tests, native mocking (e.g., for network requests), self-healing locators, and better integration with Appium. Most of these features are related to testing. For that reason, they are not covered in Selenium, which is not a testing framework but a general-purpose browser automation library.
- Improvements in the existing Selenium API, such as multi-tab and multi-window handling, shadow DOM element finder, enhanced

page factory, and improved explainability of error messages. Additionally, several users requested improvements related to WebDriver BiDi [23]. This W3C protocol enables bidirectional communication with browsers, supporting real-time event streaming and interactive features such as network monitoring and debugging. Specifically, some respondents requested WebDriver BiDi Typescript typings and a generalized BiDi feature to remove the dependency on browser-specific versions (Chrome DevTools Protocol[34]) to ensure long-term compatibility and reduce code changes after Selenium updates.

- Features that are already available in Selenium. Specifically, automated browser management (already possible with Selenium Manager), handling table elements (already possible with XPath), and full-page screenshots and network interception (already possible with WebDriver BiDi).

RQ2 Summary: There is no clear consensus on the most relevant challenges related to Selenium-based development, as all the well-known typical problems (e.g., Assertability, Asynchrony, Brittleness, Failure Analysis, Flakiness, Maintainability, Scalability, and Slowness) can occur depending on the specific testing practices. Common strategies to solve these problems include explicit waiting, robust and stable locators, the POM design pattern, and specific tooling. Practitioners would find built-in testing features desirable in the Selenium API; however, these are missing because Selenium was not designed as a testing framework but rather as a general-purpose browser automation library.

3.4. RQ3. AI-assisted tools

Fig. 19 shows the results of item D01, aimed at investigating the use of GenAI tools as an assistant in the development of Selenium-based E2E tests. This chart shows a preference for ChatGPT, with an average importance score of 3.18, indicating that it is considered moderately to highly important by respondents. GitHub Copilot follows with 2.59, also viewed as a valuable tool, though to a lesser extent. Other tools, such as Google Gemini (1.78), DeepSeek (1.27), and Amazon CodeWhisperer (1.15), received significantly lower scores, suggesting they are rarely used or valued. The "Other" category (1.42) also reflects limited importance. In this open-ended field, respondents highlighted Claude,[35] Perplexity,[36] Cursor,[37] Amazon Q,[38] Microsoft Copilot,[39] and Groq.[40]

Regarding the goal of using AI in Selenium-based development (item D02), we studied the list of the most relevant software testing facets supported by AI provided in [5]. Fig. 20 depicts the results for this question, highlighting that test case generation is considered the most relevant application, with an average score of 2.6, indicating occasional to moderate use. All other goals, such as test case maintenance (1.85), exploratory testing (1.76), and self-healing tests (1.67), received lower scores, suggesting they are perceived as less impactful or are less commonly adopted. Lower still are areas like visual testing (1.57), test oracle generation (1.51), and performance testing (1.35). The "Other" category is also very scarce (1.32), highlighting the use of AI for error prediction and code review. These results suggest that the community currently views AI primarily as a support tool for creating test cases rather than for more advanced or deeper testing tasks.

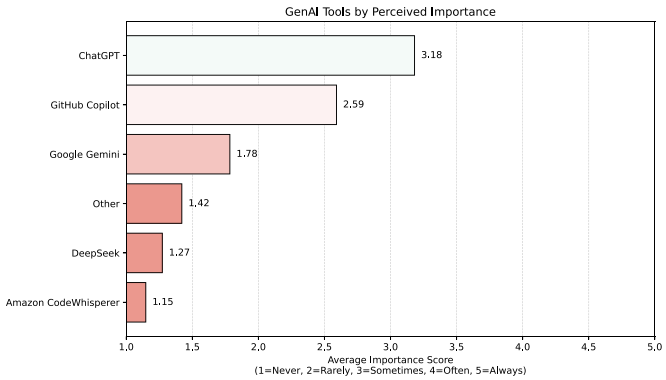


Fig. 19. AI-Assisted tools for Selenium Results: GenAI tools (D01).

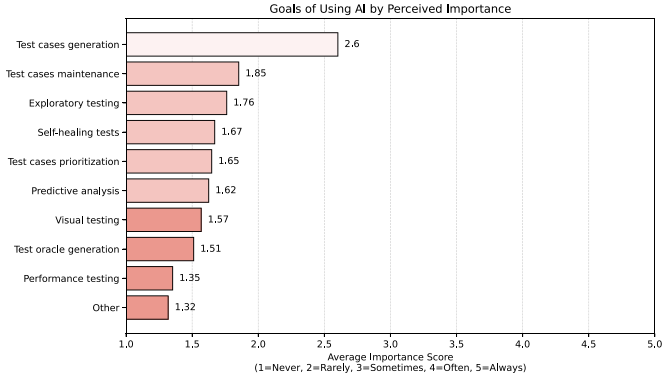


Fig. 20. AI-Assisted tools for Selenium Results: Goals of using AI (D02).

The following item in this RQ is D03, about the use of specific AI-powered testing tools with Selenium. Fig. 21 shows the results of this item, revealing that the overall adoption and perceived value of these tools are relatively low. Applitools[41] leads with an average score of 1.58, suggesting that it is only used occasionally. Other tools, including Katalon Studio[42] (1.31), Testim[43] (1.3), and Functionize[44] (1.15), fall below that, with the lowest ratings going to Mabl[45] and Testers.ai[46] (both 1.09). Other tools chosen by the respondents (1.31) are Tabnine,[47] KaneAI,[48] TestSigma,[49] Nogrunt,[50] Browser Use,[51] and testRigor.[52] These results suggest that, while the number of specific AI tools for software testing is increasing, they have yet to gain significant traction or be widely adopted as essential in most workflows.

The final item of this RQ (D04) examines the top concerns practitioners have when integrating AI into their workflows. The results, as illustrated in Fig. 22 shows that the most significant issue is over-reliance on AI (3.12), reflecting fears that excessive dependence could erode critical thinking and problem-solving skills. This problem is closely followed by privacy concerns (3.05) and security vulnerabilities

[34] https://chromedevtools.github.io/devtools-protocol/.
[35] https://claude.ai/.
[36] https://www.perplexity.ai/.
[37] https://www.cursor.com/.
[38] https://aws.amazon.com/q/.
[39] https://copilot.microsoft.com/.
[40] https://groq.com/.
[41] https://applitools.com/.
[42] https://katalon.com/.
[43] https://www.testim.io/.
[44] https://www.functionize.com/.
[45] https://www.mabl.com/.
[46] https://testers.ai/.
[47] https://www.tabnine.com/.
[48] https://www.lambdatest.com/kane-ai.
[49] https://testsigma.com/.
[50] https://www.nogrunt.com/.
[51] https://browser-use.com/.
[52] https://testrigor.com/.

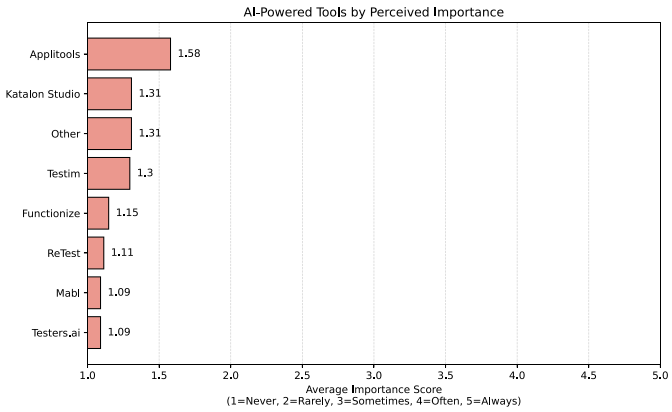


Fig. 21. AI-Assisted tools for Selenium Results: AI-Powered testing tools (D03).

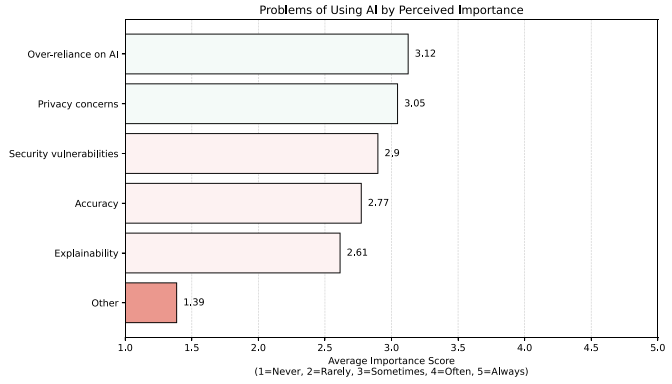


Fig. 22. AI-Assisted tools for Selenium Results: Problems of using AI (D04).

(2.9), both of which relate to the risks of sharing sensitive or proprietary data with AI systems. Accuracy (2.77) and explainability (2.61) are also notable, highlighting concerns about false positives and the difficulty of understanding AI-generated results. The "Other" category scores low (1.39). However, this field captures additional problems, such as inconsistency (i.e., unreliable or non-repeatable results), obsolete responses (i.e., generation of outdated or incompatible test code that fails to run or follow current best practices), bias in suggestions (i.e., AI favoring certain tools, patterns, or practices based on its training data), and the need for prompt engineering skills (i.e., crafting precise, unambiguous instructions to guide the AI effectively).

RQ3 Summary: ChatGPT and GitHub Copilot are the most popular AI tools for Selenium developers, primarily used to facilitate the generation of test cases. Although the landscape of AI-powered tools is expanding, they have not yet been widely adopted. Over-reliance on AI and privacy concerns are identified as the most frequent issues associated with its use.

3.5. RQ4. Alternative tools

The last RQ investigates alternatives to Selenium for E2E testing. Hence, the results of item E01 are shown in Fig. 23, which indicates that among the alternative tools to Selenium, Playwright[53] stands out with a significantly higher average importance score of 2.85, suggesting that it is used frequently and is perceived as a strong emerging tool.

[53] https://playwright.dev/.

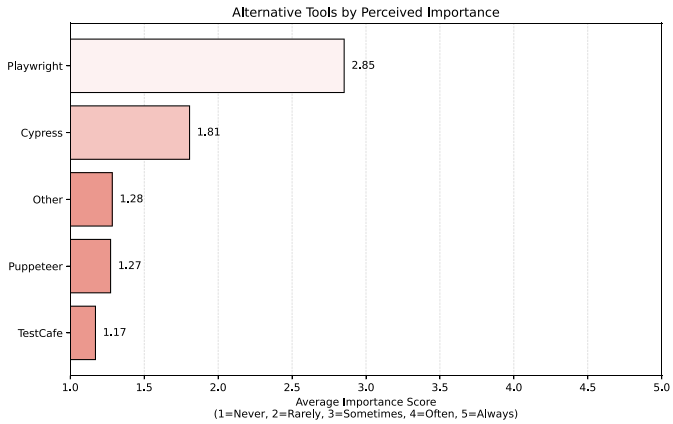


Fig. 23. Alternative tools to Selenium Results (E01).

Then, Cypress[54] follows at 1.81, indicating a more limited adoption. Other tools, such as Puppeteer[55] (1.27) or TestCafe[56] (1.17), have notably low scores, suggesting that they are rarely used or considered less relevant by the respondents. Then, the "Other" category is also low (1.28), but it provides ideas about other E2E tools, such as Appium,[57] SikuliX,[58] and WebdriverIO.[59] It should be noted that WebDriverIO can be considered as a part of the Selenium ecosystem, as it relies on the WebDriver protocol for browser automation [24].

Then, items E02 and E03 study the pros and cons of these alternative tools compared to Selenium. These questions were designed as open-ended fields, and the gathered answers are summarized in Table 5. Several respondents agreed that it is difficult to compare Selenium (a browser automation library) with tools like Playwright or Cypress (E2E testing frameworks), as they are fundamentally different tools. Also, although E2E testing frameworks provide specific built-in features for testing, some respondents acknowledged the value of Selenium as an entirely open-source project with a well-established governance and project structure. Lastly, in addition to the specific inconveniences depicted in Table 5, the common inconveniences of the alternative tools to Selenium are less community support, a smaller ecosystem, and fewer job opportunities.

Lastly, E04 analyzed the reasons respondents gave for stopping Selenium use. Overall, many respondents agreed that Selenium lacks the specific testing features they typically require, and consequently, creating a custom testing framework on top of Selenium is a common practice. Other reasons for leaving Selenium were:

- The learning curve (too high for beginners).
- Job requirements (i.e., when employers start demanding other tools).
- Difficulty of use (if the maintenance of Selenium-based projects becomes too complicated).
- Missing required features in modern web apps (e.g., shadow DOM).
- If the Selenium project stops being maintained, becomes obsolete, or if it stops being open-source.
- Performance compared with other tools (e.g., Playwright).
- Vendor pressure (incentives to switch).

[54] https://www.cypress.io/.
[55] https://pptr.dev/.
[56] https://testcafe.io/.
[57] https://appium.io/.
[58] https://sikulix.github.io/.
[59] https://webdriver.io/.

Table 5
Benefits and inconveniences of alternative tools compared to selenium (E02 and E03).

Tool	Benefits	Inconveniences
Playwright	- Fast execution due to direct WebSocket communication with the browser. - Easy setup. - Automatic waits. - Network mocking. - Mobile emulation. - Simple API. - Advanced features, such as the trace viewer and time machine. - Rich documentation. - Built-in logic for component testing.	- Do not use real browsers but patched releases of Chromium, Firefox, and WebKit. - Fake mobile browser automation.
Cypress	- Fast execution due to direct test execution within the browser. - Easy setup (it works out of the box with minimal configuration). - Automatic waits. - Live test debugging. - Built-in assertions. - Built-in screenshot and screencasting. - Integrated tools for running and debugging tests: Test Runner (Graphical Line Interface -GUI-) and CLI (Command Line Interface). - Network interception/mocking.	- Parallel execution requires Cypress Cloud (not free). - Not multi-language (only JavaScript is supported).
Puppeteer	- Better integration with npm and Chrome compared to Selenium. - Good support for web scraping.	- Limited cross-browser support.
SikuliX	- Visual-based locators and oracles.	- It tends to be slow. - Not well-documented.

RQ4 Summary: E2E testing frameworks, such as Playwright and Cypress, offer developer-friendly APIs with built-in testing features compared to Selenium, including automatic waiting, network interception, fast execution, and easy setup. On the other hand, these frameworks also have drawbacks, such as not driving real browsers but patched releases (in the case of Playwright) and not being multilingual or entirely free (in the case of Cypress).

3.6. Other aspects

In the last section of the survey, respondents were asked to provide any further comments about Selenium or any other topic (item F01). Although this question was optional, many respondents highlighted the value of Selenium as a well-established, stable, open-source project with high value for companies and practitioners. Some final recommendations for the future of Selenium emphasized the need for an easy-to-use, official Selenium-based framework and an MCP (Model Context Protocol) server. An MCP server is a communication infrastructure gaining popularity in the context of AI, facilitating efficient, scalable, and low-latency communication between multi-agent systems [25].

Finally, item F02 collected the email addresses of respondents who requested a copy of the survey findings, with more than half (56%) doing so.

4. Discussion

This section discusses the survey results, synthesizing the findings to explain underlying causes and identify broader implications for research and practice.

4.1. Selenium usage practices (RQ1)

The results of RQ1 confirm that Selenium remains a central component of industrial E2E testing, particularly for regression and functional testing. The way Selenium is used reveals a strong reliance on established, stability-oriented practices, such as local browser execution, explicit waiting strategies, ID- and XPath-based locators, and the POM. This convergence around a relatively narrow set of practices suggests a form of community-level standardization. Practitioners favor techniques that are well understood, widely documented, and perceived as robust.

Regarding programming language, our survey found that Java dominates among respondents (over 70%), followed by Python and JavaScript, reflecting a strong bias toward Java in professional QA environments. However, anonymous usage data collected by Selenium Manager[60] (via Plausible analytics) paints a different picture: based on the past five stable Selenium releases, Python is currently the most widely used language, followed by C# and Java. This divergence can arise from two key factors. First, there could be sampling bias, as our survey relies on convenience and snowball sampling, which draws heavily from established QA teams where Java is still the norm. Second, due to the telemetry scope, since Selenium Manager metrics capture real-world usage across all bindings, including open-source and smaller-scale projects, where Python has a lower barrier to entry, it is driving broader adoption. Together, these insights suggest a dual ecosystem: enterprise-grade Selenium usage remains centered on Java, while community and smaller projects favor Python.

Overall, RQ1 suggests that Selenium's longevity is not primarily due to innovation at the API level, but instead to its predictability, flexibility, and compatibility with existing toolchains. These characteristics help explain why Selenium remains dominant despite increasing competition from more integrated E2E frameworks.

4.2. Selenium challenges (RQ2)

RQ2 reveals that long-standing challenges (particularly assertability, asynchrony, and brittleness) remain highly relevant despite widespread awareness of best practices. This persistence indicates that these challenges are not merely the result of poor test design but stem from structural mismatches between Selenium's design goals and its predominant use as a testing tool.

Selenium was conceived as a general-purpose browser automation library rather than a testing framework. As a consequence, core testing concerns (such as expressive assertions, synchronization abstractions, observability, and failure diagnosis) are delegated mainly to user code or external tools. The prominence of assertability as the top-ranked challenge reflects this limitation: modern web applications increasingly rely on asynchronous behavior, dynamic rendering, and visual feedback, which are difficult to validate with DOM-centric assertions alone. Similarly, the continued relevance of asynchrony and brittleness suggests that even disciplined use of explicit waits, stable locators, and POM cannot fully compensate for the inherent coupling between test logic and evolving UI implementations. The qualitative responses reinforce this interpretation, as practitioners frequently mention workarounds (custom waits, visual testing tools, resilient locators) rather than definitive solutions.

Notably, the absence of a single dominant challenge across respondents highlights the context-dependent nature of Selenium problems. Test stability and maintainability are shaped not only by tooling, but also by application architecture, team practices, and test suite scale. From a research perspective, these findings indicate that challenges in Selenium-based testing arise from interactions among tools, practices, and development contexts, rather than from isolated technical shortcomings.

[60] https://plausible.io/manager.selenium.dev.

4.3. AI adoption in Selenium (RQ3)

The findings of RQ3 show that AI adoption in Selenium-based testing is currently goal-driven rather than tool-driven. While respondents report using a variety of AI tools, the more durable insight lies in why practitioners choose to use AI and which testing activities they are willing to support with it.

Test case generation emerges as the primary goal of AI use, indicating that practitioners perceive the greatest value in AI's ability to reduce manual effort during early test development. This task is relatively well-bounded, carries limited risk, and allows developers to retain control over validation and interpretation. In contrast, more autonomous AI applications (such as test oracle generation, predictive analysis, or self-healing tests) are less widely adopted in practice, as they are perceived as more critical and risk-prone.

This pattern suggests that current AI adoption is incremental. Practitioners are comfortable using AI as a productivity aid, but remain cautious about delegating decision-making responsibilities that could affect test correctness or accountability. Concerns about accuracy, explainability, or privacy further reinforce this interpretation and help explain the limited adoption of specialized AI-powered testing platforms. This observation motivates further research into how trust in AI-generated test artifacts develops over time, and how tool design can better support human-in-the-loop testing workflows.

4.4. Alternatives to Selenium (RQ4)

The results of RQ4 indicate that Playwright is increasingly seen as a viable alternative to Selenium, while other tools receive less attention. This trend addresses many of the challenges identified in RQ2: newer frameworks offer tighter integration, built-in synchronization, richer assertions, and an improved developer experience.

However, the coexistence of Selenium and alternative tools suggests that tool replacement is not a binary decision. Instead, practitioners appear to evaluate E2E tools based on trade-offs between maturity, ecosystem support, flexibility, and abstraction level. Selenium's continued relevance, despite acknowledged limitations, indicates that its strengths (stability, cross-browser support, and ecosystem compatibility) still outweigh its drawbacks in many contexts.

From a research standpoint, this coexistence highlights the need for comparative studies that move beyond feature checklists and instead evaluate how different tools influence test design, maintenance effort, and long-term test suite evolution.

4.5. Research and practical implications

Collectively, the findings of this study suggest that many challenges and trends in Selenium-based testing reflect broader tensions in E2E testing, such as automation versus control, abstraction versus observability, and productivity versus trust. Addressing these tensions requires not only new tools but also new abstractions, empirical models, and evaluation methods.

For researchers, the results motivate future work on higher-level testing abstractions, adaptive synchronization mechanisms, AI-assisted testing workflows, and longitudinal studies of test suite evolution. For practitioners and tool builders, the findings highlight the importance of transparency, explainability, and incremental adoption paths when introducing new automation or AI-based solutions.

5. Threats to validity

As with any empirical study, it is paramount to assess the validity of our findings critically. In this section, we analyze potential threats across four commonly accepted dimensions: internal validity, which concerns the reliability of our conclusions; external validity, which concerns the generalizability of our results; construct validity, which concerns the appropriateness and clarity of the survey measures used; and conclusion validity, which concerns the extent to which the observed results support the relationships and inferences drawn from the collected data.

5.1. Internal validity

To minimize bias in survey design and interpretation, we conducted pilot tests with domain experts and iteratively refined the questionnaire to ensure its accuracy. Nonetheless, internal validity may be affected by the subjective interpretation of open-ended responses. We analyzed open-ended responses using thematic coding, a qualitative method that involves labeling text segments with descriptive codes and grouping them into higher-level themes [11]. While this method adds structure and consistency to qualitative analysis, it remains subject to researcher interpretation. Furthermore, the lack of incentives may have introduced self-selection bias, leading only motivated participants to respond.

5.2. External validity

The generalizability of our findings is limited by the nonprobabilistic sampling methods used. While we invited a broad group of practitioners via conferences, social media, and professional contacts, we had no control over who saw the invitation or how widely it was shared. Although this approach helped reach a specialized population, it reflects the characteristics of convenience and snowball sampling rather than random sampling. Consequently, the results may not accurately represent the full diversity of Selenium users, particularly those who are not active in professional networks or conferences.

5.3. Construct validity

Construct validity concerns may arise due to the self-reported nature of survey responses. Participants may have interpreted terms (e.g., "Flakiness" or "Assertability") differently depending on their background and experience. We attempted to mitigate this risk by basing our terminology on prior validated studies [3–6] and offering examples where appropriate. Also, we incorporated open-ended fields in most survey items to enhance construct validity. This design choice enabled respondents to elaborate on their choices, describe additional tools or practices not included in the predefined options, and provide context-specific feedback, thereby ensuring that the survey captured a broader, more accurate representation of practitioners' experiences and concerns.

5.4. Conclusion validity

Conclusion validity concerns the extent to which the collected data and the applied analysis support the conclusions drawn in the study. A primary threat in our work stems from the survey's and analysis's descriptive nature. As the study relies on descriptive statistics rather than inferential testing, the reported results should be interpreted as indicative trends within the observed sample, rather than as statistically validated differences or causal relationships.

Additionally, several findings are derived from Likert-type scales, for which averages may mask response variability. To mitigate this risk, we complement mean values with medians and distribution plots for key items, reducing the likelihood of over-interpreting small numerical differences. Finally, conclusions drawn from open-ended questions depend on thematic coding and synthesis, which may introduce researcher bias. This threat is mitigated by applying a systematic coding process and by providing a replication package that enables transparency and independent verification.

6. Related work

Several studies and industrial reports reflect the central role of Selenium in modern test automation workflows [2,26]. Previous work

by García et al. [6] surveyed the Selenium ecosystem, detailing its components, tools, and industry adoption. This study highlighted key use cases, such as cross-browser testing and CI/CD integration, while identifying common challenges like flaky tests and maintenance complexity. Similarly, Garousi et al. [27] analyzed 142 primary studies to evaluate trends, benefits, and challenges in automated testing tools, identifying Selenium as one of the most widely used tools and emphasizing its role in automated web regression testing. The study highlights key adoption factors (e.g., ease of integration, community support) and common limitations, such as flakiness and maintenance overhead. Then, Leotta et al. [3] systematically examined common problems in Selenium-based E2E testing through a survey of practitioners, identifying the top 13 challenges, including handling asynchronous behavior, test flakiness, and brittleness due to evolving web UIs, slow execution times, maintenance overhead, cross-browser compatibility, driver infrastructure management, and difficulty in failure analysis. While these studies provide valuable baseline insights into Selenium usage and challenges, they either focus on specific aspects (e.g., ecosystem components or isolated problem categories) or reflect practices observed before recent shifts in tooling and development practices. In contrast, our study revisits these topics with updated industrial data and integrates usage patterns, challenges, AI adoption, and alternative tools within a single empirical framework.

Other studies (e.g., [4]) compared Selenium with newer tools such as Cypress, Puppeteer, and Playwright, highlighting their architectures, feature sets, and trade-offs. The study finds that Selenium, with its standard foundations based on W3C WebDriver [24], supports multiple languages and full cross-browser support, but it suffers from flakiness and slower execution. Similarly, [28] examined other open-source automation testing tools (Selenium, Katalon Studio, and Test Project) and evaluated them across usability, language support, setup complexity, built-in features, parallel execution, reporting, and maintainability. This study found that Selenium offers extensive flexibility and multi-language/browser support but requires significant setup and coding expertise. Unlike these comparative studies, which primarily analyze tool features and architectural differences, our work captures practitioners' perceptions and decision-making criteria, shedding light on why and under which conditions Selenium continues to be used alongside (or replaced by) newer E2E frameworks.

Recent advances in AI have significantly enhanced Selenium-based E2E testing, addressing its inherent challenges while expanding its capabilities [5]. For example, Khaliq et al. introduced a deep learning-driven framework that automates the generation and execution of Selenium functional tests for e-commerce applications by combining object detection, text recognition, and natural language models. The proposed framework automatically extracts the UI structure, transforming it into executable test scripts using a parser, achieving 98.8% accuracy in generating correct tests and maintaining 98.6% accuracy even after UI modifications. Leotta et al. [29] explored the effectiveness of ChatGPT and GitHub Copilot in generating Selenium-based E2E test scripts from Gherkin-style specifications across eight real-world web applications. The results of this study showed that using ChatGPT reduces test generation time by about 30% compared to manual scripting, while Copilot offers similar productivity gains. Whereas existing work evaluates the effectiveness of specific AI techniques or tools in controlled settings, our study complements these efforts by examining how practitioners currently use AI in practice, with a particular emphasis on the goals and intentions driving adoption rather than on individual tools.

7. Conclusions and future work

This study presents the results of a comprehensive survey aimed at understanding how Selenium is used for E2E testing in practice, the challenges users face, the role of AI-assisted tools, and the adoption of emerging alternatives. By analyzing responses from 88 practitioners, we provide a snapshot of the current Selenium ecosystem and the evolving testing practices within it.

Our findings confirm the central role of Selenium in E2E test automation, primarily used for regression and functional testing. Nevertheless, recurring challenges in Selenium-based testing (particularly around assertability, asynchrony, brittleness, and failure analysis) persist, highlighting the fragility of tests in dynamic, real-world environments. Robust coding strategies, including explicit waits, stable locators, and reusability through POM design patterns, are well-known best practices for avoiding common pitfalls.

The survey also reveals a growing interest in AI-assisted tools. ChatGPT and GitHub Copilot are the most commonly used GenAI solutions, primarily aiding in test case generation. However, concerns around over-reliance, data privacy, and explainability remain significant barriers to widespread adoption. Similarly, while Playwright and Cypress are gaining traction due to their built-in testing capabilities, Selenium remains valued for its flexibility, cross-browser/language support, robust community, and rich ecosystem.

Future work will pursue several directions. First, we plan to replicate this survey with a broader and more diverse participant base to mitigate sampling bias and validate trends over time. Second, we aim to conduct longitudinal studies to track the adoption of AI tools and emerging frameworks like Playwright. Third, we see a need for empirical evaluations of AI-generated test artifacts and their real-world effectiveness. Finally, our findings suggest an opportunity for future development in the Selenium ecosystem to provide built-in native support for test-specific features (e.g., automatic waits or visual assertions).

CRediT authorship contribution statement

Boni García: Writing – review & editing, Writing – original draft, Resources, Methodology, Investigation, Data curation, Conceptualization. **Filippo Ricca:** Writing – review & editing, Supervision, Methodology, Funding acquisition. **Maurizio Leotta:** Writing – review & editing, Methodology, Data curation, Conceptualization. **Mario Munoz-Organero:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work has been partially supported by the "Generation of Reliable Synthetic Health Data for Federated Learning in Secure Data Spaces" Research Project, Spain (PID2022-141045OB-C43 (AEI/ERDF, EU)) funded by MCIN/AEI/10.13039/501100011033 and by "ERDF A way of making Europe" by "the European Union", and by the MICIU/AEI/ 10.13039/501100011033, Spain and by ERDF/UE through the GENIELearn Research Project, Spain (PID2023-146692OB-C31).

This study was partially carried out within the "EndGame - Improving End-to-End Testing of Web and Mobile Apps through Gamification" project, Italy (2022PCCMLF) – Next Generation EU within the PRIN 2022 program (D.D.104 - 02/02/2022 Ministero dell'Università e della Ricerca). This manuscript reflects only the authors' views and opinions, and the Ministry cannot be considered responsible for them.

Data availability

The research data is public in a GitHub repository (linkend in the paper)

References

[1] B.A. Kitchenham, S.L. Pfleeger, Personal opinion surveys, in: Guide To Advanced Empirical Software Engineering, Springer, 2008, pp. 63–92.

[2] M. Cerioli, M. Leotta, F. Ricca, What 5 million job advertisements tell us about testing: a preliminary empirical investigation, in: Proceedings of the 35th Annual ACM Symposium on Applied Computing, 2020, pp. 1586–1594.

[3] M. Leotta, B. García, F. Ricca, J. Whitehead, Challenges of end-to-end testing with Selenium WebDriver and how to face them: A survey, in: The 16th IEEE Conference on Software Testing, Verification and Validation, ICST, 2023, pp. 339–350.

[4] B. García, J.M. del Alamo, M. Leotta, F. Ricca, Exploring browser automation: A comparative study of Selenium, Cypress, Puppeteer, and Playwright, in: Quality of Information and Communications Technology, Springer Nature Switzerland, Cham, 2024, pp. 142–149.

[5] F. Ricca, B. García, M. Nass, M. Harman, Next-generation software testing: AI-Powered test automation, IEEE Softw. 42 (4) (2025) 25–33.

[6] B. García, M. Gallego, F. Gortázar, M. Munoz-Organero, A survey of the Selenium ecosystem, Electronics 9 (7) (2020).

[7] T. Punter, M. Ciolkowski, B. Freimut, I. John, Conducting on-line surveys in software engineering, in: 2003 International Symposium on Empirical Software Engineering, 2003. ISESE 2003. Proceedings, IEEE, 2003, pp. 80–88.

[8] A. Jedlitschka, M. Ciolkowski, C. Denger, B. Freimut, A. Schlichting, Relevant information sources for successful technology transfer: a survey using inspections as an example, in: First International Symposium on Empirical Software Engineering and Measurement, ESEM 2007, IEEE, 2007, pp. 31–40.

[9] B. García, M. Munoz-Organero, C. Alario-Hoyos, C.D. Kloos, Automated driver management for Selenium WebDriver, Empir. Softw. Eng. 26 (5) (2021) 107.

[10] M. Leotta, B. García, F. Ricca, A family of experiments to quantify the benefits of adopting WebDriverManager and Selenium-Jupiter, Inf. Softw. Technol. 178 (2025) 107595.

[11] M. Williams, T. Moser, The art of coding and thematic exploration in qualitative research, Int. Manag. Rev. 15 (1) (2019) 45–55.

[12] T.C. Lethbridge, A survey of the relevance of computer science and software engineering education, in: Proceedings 11th Conference on Software Engineering Education, IEEE, 1998, pp. 56–66.

[13] B. García, Hands-On Selenium WebDriver with Java, O'Reilly Media, Inc., United States, 2022.

[14] K. Srinath, Python–the fastest growing programming language, Int. Res. J. Eng. Technol. 4 (12) (2017) 354–357.

[15] K. Presler-Marshall, E. Horton, S. Heckman, K. Stolee, Wait, Wait. No, Tell Me. Analyzing Selenium configuration effects on test flakiness, in: 2019 IEEE/ACM 14th International Workshop on Automation of Software Test, AST, 2019, pp. 7–13.

[16] D. Olianas, M. Leotta, F. Ricca, Leveraging Large Language Models for Explicit Wait Management in End-to-End Web Testing, in: 2025 IEEE Conference on Software Testing, Verification and Validation, ICST, 2025, pp. 577–581.

[17] M. Leotta, D. Clerissi, F. Ricca, C. Spadaro, Comparing the maintainability of Selenium webdriver test suites employing different locators: A case study, in: Proceedings of the 2013 International Workshop on Joining Academia and Industry Contributions To Testing Automation, 2013, pp. 53–58.

[18] M. Hu, A. Trofimov, Course design of introducing Selenium WebDriver, in: 2024 IEEE International Conference on Software Testing, Verification and Validation Workshops, ICSTW, 2024, pp. 340–348.

[19] D. Kovalenko, Selenium Design Patterns and Best Practices, Packt Publishing Ltd, 2014.

[20] M. Leotta, D. Clerissi, F. Ricca, C. Spadaro, Repairing Selenium test cases: An industrial case study about web page element localization, in: 2013 IEEE Sixth International Conference on Software Testing, Verification and Validation, IEEE, 2013, pp. 487–488.

[21] B. García, Introducing Selenium Manager, 2022, https://www.selenium.dev/blog/2022/introducing-selenium-manager/, [Online; (Accessed 12 June 2025)].

[22] J.F. Smart, J. Molak, BDD in Action: Behavior-Driven Development for the Whole Software Lifecycle, Simon and Schuster, 2023.

[23] J. Graham, A. Rudenko, M. Sadym, WebDriver BiDi. Editor's Draft, 3 June 2025, 2025, Available at: https://w3c.github.io/webdriver-bidi/. (Accessed 18 June 2025).

[24] S. Stewart, D. Burns, WebDriver, W3C working draft 12 May 2025, 2025, Available at: https://www.w3.org/TR/webdriver/. (Accessed 16 June 2025).

[25] X. Hou, Y. Zhao, S. Wang, H. Wang, Model context protocol (mcp): Landscape, security threats, and future research directions, 2025, arXiv preprint arXiv:2503.23278.

[26] F. Ricca, A. Stocco, Web test automation: Insights from the grey literature, in: SOFSEM 2021: Theory and Practice of Computer Science: 47th International Conference on Current Trends in Theory and Practice of Computer Science, SOFSEM 2021, Bolzano-Bozen, Italy, January 25–29, 2021, Proceedings 47, Springer, 2021, pp. 472–485.

[27] L. Prasad, R. Yadav, N. Vore, A systematic literature review of automated software testing tool, in: Proceedings of 3rd International Conference on Computing Informatics and Networks: ICCIN 2020, Springer, 2021, pp. 101–123.

[28] B. Majeed, S.K. Toor, K. Majeed, M.N.A. Chaudhary, Comparative study of open source automation testing tools: Selenium, Katalon Studio & test project, in: 2021 International Conference on Innovative Computing, ICIC, IEEE, 2021, pp. 1–6.

[29] M. Leotta, H.Z. Yousaf, F. Ricca, B. Garcia, AI-generated test scripts for web E2E testing with ChatGPT and Copilot: A preliminary study, in: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, EASE '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 339–344.